Relatório de 1º Projeto: Computação Paralela

Rúben Silva
raf.silva@campus.fct.unl.pt

1 INTRODUÇÃO

Hoje em dia, a utilização de ferramentas de IA para gerar código de pipelines de Spark em minutos é cada vez mais comum. Contudo, o código gerado raramente vem otimizado, deste modo, cabe a nós de forma minuciosa e sistemática analisar métricas associadas ao pipeline de modo a encontrar bottlenecks e os resolver da maneira mais apropriada. Já que estes podem impactar significativamente a performance.

Para este projeto foram utilizados dataset real da NYC Taxi Limousine Commission (TLC), focado nos registos mensais relativos às viagens de táxi amarelo do ano 2025, complementados pelo ficheiro de mapeamento de zonas.

O pipeline implementa três operações obrigatórias, M1, recolha e limpeza de dados, M2, análise da procura ao longo do tempo e M3, estatísticas ao nível da zona. Adicionalmente, foram implementadas de dentro de quatro operações opcionais duas, sendo as selecionadas O1, detecção de anomalias nas tarifas, e O4, classificação por tipo de viagem, cujos custos computacionais justificam a nossa análise de desempenho.

2 ARQUITETURA DO PIPELINE E PLANO DE EXECUÇÃO FÍSICOS

O pipeline foi construído utilizando a API de DataFrames do PySpark para tirar pleno partido das otimização automática de desempenho e da gestão de memória eficiente.

2.1 Decisões Arquiteturais

- **API DataFrame vs. Spark SQL:** Optou-se pela API de DataFrames para estruturar transformações de forma modular, garantir verificação de colunas em tempo de compilação e aplicar *hints* de otimização explicitamente (`.hint("broadcast")`).
- **Conjunto de Dados Base Centralizado:** A limpeza e derivação inicial de atributos (ex.: `trip_duration_mins`, extrações de hora e dia) são realizadas apenas uma vez em **M1**. As operações subsequentes (**M2**, **M3**, **O1**, **O4**) consomem este DataFrame partilhado.

3 PERFILAMENTO E BASELINE

3.1 Configuração Experimental

Os ensaios foram realizados num *cluster* constituído por 1 nó Driver e 3 nós Worker:

- **Versão do Spark:** Apache Spark 3.x via PySpark.
- **Recursos do Cluster:** 3 nós Worker, memória de driver configurada para 2GB.
- **Configurações ATIVAS:** `spark.sql.adaptive.enabled = true`, `spark.sql.adaptive.skewJoin.enabled = true`.

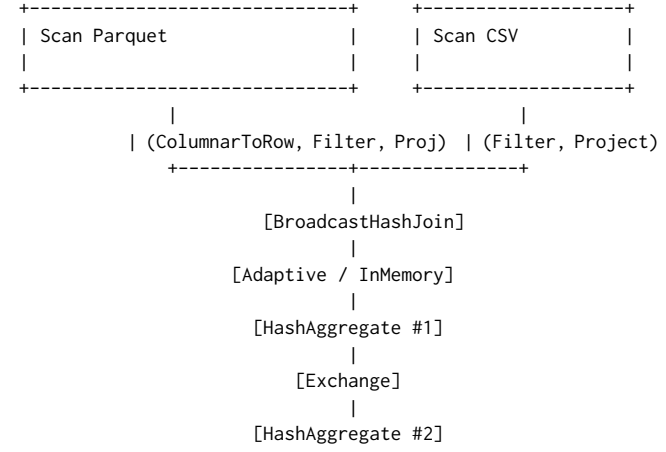


Figure 1: Fluxo de Execução Simplificado de M1

3.2 Identificação dos Gargalos Iniciais

A execução do código base (*baseline*) juntamento com o programa dado "spark profiler", permitiu diagnosticar quatro gargalos principais:

(1) **Custo de Inferência de Esquema:** A leitura do CSV com inferência adicionou tarefas e *jobs* extra desnecessários antes de processar os ficheiros Parquet.
(2) **Overhead de Cache em Tabelas Pequenas:** Tentar fazer *cache* do ficheiro `taxi_zone_lookup.csv` (263 linhas) aumentou o tempo total em 22.1%, pois o custo de gerir a memória superou o tempo de releitura do disco.
(3) **Armazenamento de Dados Não Podados:** Guardar em *cache* a tabela completa resultante de **M1** provocou uma inflação no volume de dados lidos (2.5 GB a 4.5 GB) devido à preservação de colunas não utilizadas.
(4) **Volume Elevado de Shuffles:** As operações **O1** e **M3** criavam elevado número de tarefas e dependências entre *stages* devido a múltiplas definições de janelas independentes.

4 ESTUDO DE OTIMIZAÇÃO

Adotou-se o ciclo metodológico: *perfilamento* → *hipótese* → *otimização* → *medição*. Uma nota importante é que a ordem que é apresentadas as otimizações foi a ordem de implementação e as medições que foram retirando foram acumolando essas otimizações (ex.: na otimização 4.4 as medições são comparadas com o pipeline com otimizações até 4.3)

4.1 M1: Eliminação da Inferência de Esquema

- **Gargalo Observado:** O leitor base utilizava `inferSchema="true"`, gerando 6 *jobs* e 7 *stages*.

- **Hipótese:** Passar um esquema `StructType` explícito elimina a tarefa preliminar de leitura do ficheiro.
- **Resultados:** O tempo de execução (*wall time*) caiu de 4.83 s para 2.99 s (−38.1%), reduzindo o número de *jobs* de 6 para 4 (−33.3%) e o tempo de CPU em 25.5%.

4.2 Caso de Otimização Negativa: Cache da Tabela de Zonas

- **Gargalo Observado:** Colocar a tabela de zonas em *cache* aumentou o tempo total de 2.76 s para 3.37 s (+22.1%) e os *jobs* de 4 para 7 (+75.0%).
- **Análise:** O *overhead* de registar e gerir um bloco de memória para 263 linhas é superior à leitura direta do ficheiro em disco. A *cache* é contraproducente em dados irrisórios.

4.3 M1: Podagem de Colunas Antes da Cache (Column Pruning)

- **Gargalo Observado:** Fazer *cache* da tabela base beneficiava as operações seguintes, mas consumia entre 2.5 GB e 4.5 GB de leitura devido à presença de colunas desnecessárias.
- **Otimização:** Aplicou-se um `.select()` para manter apenas as colunas essenciais antes da chamada a `.cache()`.
- **Resultados:**
  - **M2:** O tempo de execução desceu de 7.87 s para 4.58 s (−41.8%), reduzindo o volume de leitura em 55.2% (2.5 GB → 1.1 GB).
  - **O4:** O tempo caiu de 14.10 s para 7.76 s (−45.0%), com redução de 27.1% no tempo de CPU.

4.4 M3: Consolidação de Janelas e Podagem

- **Gargalo Observado:** A operação **M3** executava múltiplos *shuffles* redundantes sobre especificações de janela com a mesma chave de partição.
- **Otimização:** Reutilizou-se a mesma partição de janela e filtraram-se as colunas da tabela **M1**.
- **Resultados:** O tempo desceu de 3.27 s para 2.94 s (−7.5%), reduzindo a utilização de CPU de 11.155 ms para 10.189 ms (−8.7%).

4.5 O1: Substituição de Join por Funções de Janela

- **Gargalo Observado:** A abordagem baseline calculava a média e desvio padrão via `groupBy` e realizava um *join* de regresso, gerando 401.1 MB de *shuffle*.
- **Otimização:** Substituiu-se o padrão por funções de janela inline (`Window.partitionBy("PULocationID", "hour_of_day")`).
- **Análise de Vantagens:** Forçar um *SortMergeJoin* demorava 16.72 s, enquanto a versão com janela cumpriu em 11.23 s (−32.8%), eliminando 3 *stages* e reduzindo as tarefas de 75 para 45 (−40.0%).

4.6 Ajuste Dinâmico com Adaptive Query Execution (AQE)

A comparação da execução com AQE ligado vs. desligado revelou:

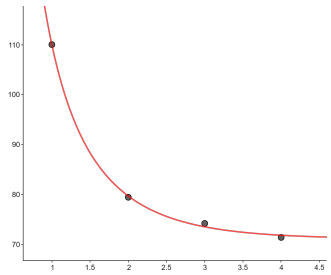


Figure 2: Curva de tempo de execução (s) em função do número de trabalhadores no *cluster*.

- **Sem AQE (M2/M3):** O Spark fixava 200 partições de *shuffle*, criando pequenas tarefas, atrasos no escalonador e maior tempo total (5.17 s no M3).
- **Com AQE Ativo:** O Spark agregou partições automaticamente, reduzindo o número de tarefas em **M2** de 210 para 21 (−90.0%) e o tempo no **M3** para 3.27 s (−36.8%).

5 ANÁLISE DE ESCALABILIDADE

5.1 Dimensionamento do Cluster (Speedup e Eficiência)

Avaliou-se o pipeline otimizado variando o número de *workers* (1, 2, 3 e 4 trabalhadores) no *cluster*. Os tempos totais de execução obtidos foram os seguintes:

- **1 Trabalhador:** 110.00 s
- **2 Trabalhadores:** 79.40 s (*speedup* de 1.39×, eficiência de 69.3%)
- **3 Trabalhadores:** 74.15 s (*speedup* de 1.48×, eficiência de 49.4%)
- **4 Trabalhadores:** 71.35 s (*speedup* de 1.54×, eficiência de 38.5%)

Como ilustrado na Figura 5.1, observa-se uma acentuada diminuição do tempo de execução ao passar de 1 para 2 trabalhadores (110.00 s → 79.40 s). No entanto, a partir dos 3 e 4 trabalhadores a curva estabiliza (74.15 s e 71.35 s), apresentando um ganho marginal (rendimentos decrescentes da Lei de Amdahl). Esta estagnação deve-se aos seguintes fatores:

(1) **Overhead de Comunicação de Rede:** O aumento do número de trabalhadores aumenta a complexidade das transferências de dados nos *shuffles* das operações de janela (**M3** e **O1**).
(2) **Fração Sequencial não Paralelizável:** Custos fixos de inicialização de tarefas pelo Driver, sincronização de estantes e escrita final em disco limitam o ganho linear ideal.
(3) **Tamanho do Conjunto de Dados vs. Recursos:** Para o volume de dados avaliado, 3 a 4 trabalhadores esgotam rapidamente o paralelismo útil das partições, fazendo com que o custo de coordenação passe a dominar a computação efetiva.

Bibliografia

(1) Apache Spark Documentation. *Spark SQL, DataFrames and Datasets Guide*. Apache Software Foundation. https://spark.apache.org/docs/latest/sql-programming-guide.html
(2) Armbrust, M., et al. (2015). *Spark SQL: Relational Data Processing in Spark*. Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, 1383–1394.
(3) New York City Taxi & Limousine Commission (TLC). *TLC Trip Record Data*. City of New York. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Utilização De IA

Foram utilizadas ferramentas de assistência por IA na fase exploratória para geração do pipeline inicial. Para além disso também foi utilizado para gerar maioria deste relatorio.